Relatório P1 - Sistemas Operacionáis II

Felipe Gassen, Felipe Murta, Guilherme Pierri, Gustavo D'Angelo

27/08/2025

1 Requisitos

Para a entrega 1 do trabalho da disciplina, foi necessário implementar a API fornecida pelo professor, permitindo com que mensagens sejam propagadas da aplicação até a placa de rede, enviadas e recebidas pela Ethernet, e depois propagadas novamente até a aplicação em outra máquina. Para essa etapa, não configuramos propagação instantânea de mensagens, fazendo uso de uma *receiver thread* para captar pacotes.

O projeto foi realizado em uma arquitetura Linux minimalista, usando o *busybox* como *shell*. Esse kernel foi compilado em um ambiente virtual simulado pelo Qemu, onde foi realizada a comunicação entre cinco VMs.

2 Implementação

Dividimos a API em diversos arquivos do tipo .hpp, contendo as *headers* das classes.

O primeiro objetivo foi implementar a classe NIC, para realizar a comunicação entre máquinas. A NIC representa uma abstração de placa de rede presente em cada veículo, enquanto a *RawSocketEngine* simula um socket puro, que captura pacotes do tipo eth0 diretamente da rede. Quando um pacote é enviado, a NIC chama a Engine que monta o frame Ethernet com os dados da mensagem e nosso número de protocolo, lançando-o como broadcast na rede. Para o recebimento do pacote, a classe NIC roda constantemente uma *reciever_thread()*, que checa se existe algum pacote na rede Ethernet. A *thread* receptora controla um semáforo, executando um algoritmo de produtor-consumidor. O semáforo começa em zero, e é incrementado quando uma mensagem é captada, enquanto a aplicação destino decrementa o semáforo. Quando é detectado um pacote, como a NIC é um objeto observado, ela ira notificar seus observadores de que um pacote foi recebido. Para esse teste de rede, foi criada a classe *test_nic*, que gera mensagens para serem enviadas pela NIC e também serve como observadora da NIC, gerando logues do tempo de saída e chegada de mensagens. Em testes posteriores, essa classe foi deteriorada em prol de mensagens que tem como origem e destino a classe *Communicator*.

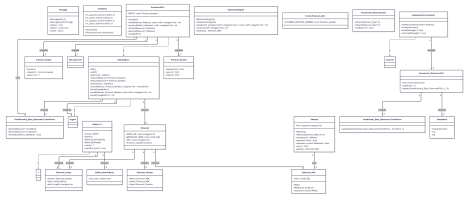

Figura 1: Diagrama de Classes do sistema implementado.

A segunda parte da implementação foi dedicada a propagação de mensagens dentro de cada VM. O fluxo da mensagem começa no Componente, onde é gerado um dado aleatório. Esse componente chama a classe *Communicator*, passando como argumento o dado que deseja enviar. *Communicator* gera uma mensagem da classe *Message* que é enviada ao *Protocol Handler*. Essa classe é responsável por adicionar o número de protocolo ao pacote e tentar alocar um *buffer* da NIC para a mensagem. A NIC então é chamada, e, assim como no caso de teste, usa a classe *RawSocketEngine* para gerar o frame Ethernet e enviá-lo.

Para subir uma mensagem captada da rede para nível de aplicação, existe uma pilha de objetos observadores e observados que notificam a chegada de uma mensagem. A implementação desses observadores é condicional, ou seja, um objeto observado pode escolher notificar somente seus observadores que se importam com dada condição. Quando a *thread* de recebimento da NIC detecta um pacote, ela chama a *RawSocketEngine* para captá-lo. A NIC é observada por uma série de observadores, os Protocolos, que tem como condição de observação seu número de protocolo. Os observadores notificados recebem um buffer contendo a mensagem captada. Para o caso da entrega P1, somente um Protocolo está observando a NIC. Cada Protocolo é observado por um número arbitrário de comunicadores, cada um referente a uma porta que servirá como condição para serem notificados pelo protocolo.

3 Resultados

Foi observada a transferência de pacotes entre dois dispositivos. Os logs de chegada dos pacotes foram demonstradas na figura 3, demonstrando a troca de mensagens entre 5 máquinas.

É possível verificar essa troca externamente fazendo uso da configuração de um Bridge e Taps ao instanciar as VMs, permitindo, assim, o uso do tcpdump para inspecionar sua rede privada através do computador hospedeiro.

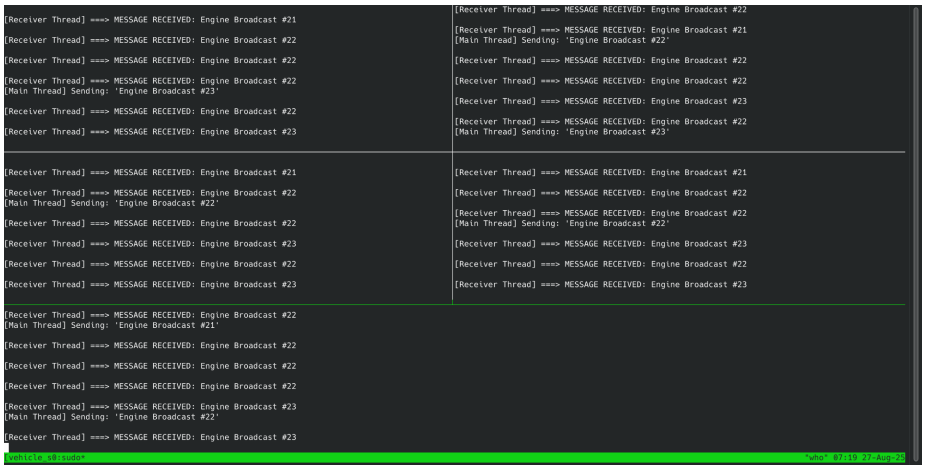

Figura 2: Logs do sistema.

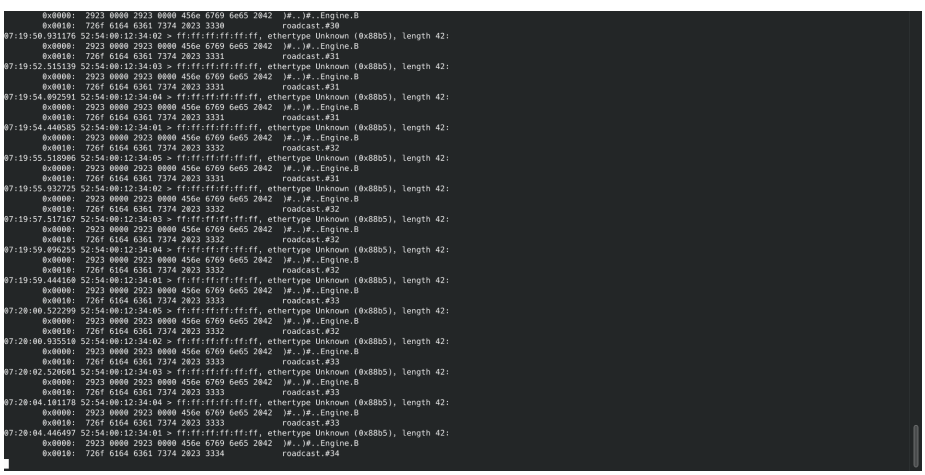

Figura 3: Logs tcpdump.